

Corporate Watch — Guided Lab

Advanced Python Lab: an aggregate competitive-intelligence monitor

Day 3 · advanced Python · you build a scheduled, organisation-level watch dashboard

You will build a monitor that tracks a target **company** from public sources: its R&D direction, its macro-corporate moves (contracts, M&A, divestitures, service stops) and an aggregate hiring-demand signal — and renders it on a dashboard. Then you point it at a **different** target and it just works.

This folder ships the reference solution. `backend/`, `frontend/` and `data/` are the finished project (the correction). Build your own first; open the solution when stuck or to compare. The guided steps below mirror the reference files.

Scope & compliance (non-negotiable)

This monitor is **organisation-level and aggregate**. It answers *"where is this company investing, what is it deciding, which domains is it growing in"* — never *"who works there"*. No individual is profiled.

`backend/common.py:drop_personal` is the last-line guard, and R&D author identities are counted then discarded. Building a name-keyed base of a company's employees is out of scope and, for a competitor, unlawful. Keep every output aggregate.

Learning objectives

- Design a modular collector pipeline (one source per module) with a common schema.
- Query keyless public APIs (OpenAlex) and RSS, and normalise their output.
- Auto-classify corporate events and tag each with an **Admiralty** source-reliability letter.
- Aggregate signals by research domain and render a dashboard from `file://`.
- Schedule the monitor and alert on change — the Day 1 pipeline discipline.

Setup

```
cd "Day_3/Corporate Watch Lab"
python3 --version          # 3.10+
# the collectors use only the standard library; jq is handy for exploring JSON
```

Part 1 — The schema & the compliance guard (20 min)

Open `backend/common.py`. Everything a collector emits is normalised into one row shape and passed through `drop_personal`.

1.1 List the fields of an event row (`event_row`). Which field carries provenance, which carries the Admiralty rating, and which is deliberately left blank for the analyst?

1.2 Read `RELIABILITY` and `reliability_for`. Map three real news domains to their letter. Why is the **information** digit (1–6) not set by the machine?

1.3 Read `drop_personal`. Write one row that it would reject and one it would keep. Why is a last-line guard worth having even when every collector is already aggregate?

Part 2 — Collector: the R&D capability map (30 min)

Goal: publication **counts** by research concept and by year, for the target company, from **OpenAlex** (keyless). Reference: `backend/sources/rnd.py`.

2.1 Find your target's OpenAlex institution id:

```
curl -s "https://api.openalex.org/institutions?search=<COMPANY>&mailto=you@example.com" \
| jq -r '.results[] | "\(.id) \(.display_name) works=\(.works_count)'"
```

2.2 Group works by concept (note: **do not** send `per_page=1`, it truncates the groups):

```
curl -s "https://api.openalex.org/works?filter=authorships.institutions.lineage:<ID>,from_publication_date:2023-01-01&group_by=concepts.id&mailto=you@example.com" \
| jq -r '.group_by[] | "\(.key_display_name)\t\(.count)'" | head -14
```

2.3 In `rnd.py`, the collector returns `{concepts, yearly}`. **Where** does it drop author identities, and why is that the whole point? Add a filter to drop absurd future years.

Part 3 — Collector: macro-corporate events (35 min)

Goal: a classified, Admiralty-rated news feed. Reference: `backend/sources/news.py` + `classify_event` in `common.py`.

3.1 Fetch the raw feed and eyeball it:

```
curl -s "https://news.google.com/rss/search?q=
<COMPANY>%20(contract%20R%20acquisition%20R%20divestiture)&hl=en-US&gl=US&ceid=US:en"
\
| grep -o '<title>[^<]*</title>' | head
```

3.2 Read `classify_event`. Add one event category with its keyword pattern (e.g. *supply chain*). Test it on five real headlines.

3.3 `news.py` also reads keyless aviation-press RSS. Add one more reputable feed and give its domain the right Admiralty letter in `RELIABILITY`.

Part 4 — Collector: aggregate hiring demand (20 min)

Goal: *how much* is the company hiring per domain — never *who*. Reference: `backend/sources/hiring.py`.

4.1 The collector counts public job-posting **mentions** per domain keyword. Add two domains relevant to your target. Confirm the output is a list of `{domain, demand_signal}` — no names.

4.2 Why is a mention **count** compliant where a scraped list of new hires is not? One sentence.

Part 5 — Orchestrate & visualise (30 min)

Reference: `backend/watch.py` (orchestrator) and `frontend/index.html` (dashboard).

5.1 Run the whole monitor against your target:

```
python3 backend/watch.py --company "<COMPANY>"
open frontend/index.html
```

5.2 `watch.py` writes `data/aggregate.json` and `frontend/data.js` (so the dashboard opens from `file://`). Explain why writing a `data.js` avoids a CORS/fetch problem on `file://`.

5.3 Change the target to a **different** company (new institution id in `rnd.py`, new `--company`). Confirm the dashboard repopulates. This is the deliverable: reusable on new data.

Part 6 — Schedule & alert (15 min)

6.1 Add a cron entry that refreshes the monitor weekly (README shows the line). Where should the working directory point, and why absolute paths?

6.2 Sketch a change-alert: diff `data/aggregate.json` between two runs and notify on new high-value events (M&A, divestiture, service-stop). Which Day 1 principle does this apply?

Deliverables

One zip `CORPWATCH_<team>.zip`:

1. Your monitor run against a company **other than the reference one**: `aggregate.json` + a dashboard screenshot.
2. Your added event category, extra press feed, and two hiring domains (diffs against the reference).
3. Answers to the numbered questions (1.1 → 6.2), including the compliance reasoning in 1.3 and 4.2.

Grading grid (100 pts)

Item	Pts	What earns the points
Schema & compliance	20	correct field roles; a valid drop_personal example; why aggregate
R&D collector	20	correct institution id, concept + yearly counts, identities dropped
Events collector	20	new category works; Admiralty letters sensible
Hiring signal	10	aggregate-only; compliance reasoning
Orchestrate & dashboard	20	runs on a new target; dashboard repopulates
Schedule & alert	10	cron + change-alert sketch

Optional Extensions (Bonus)

- **Patents** as a stronger R&D signal: add a connector (Lens.org / EPO OPS, keyed) that counts patents by CPC class for the assignee. Aggregate by class, drop inventor names.
- **Emit STIX**: have `watch.py` also produce a STIX Report + Identity + Observed Data (see the STIX Lab) so the watch feeds a shared graph.
- **De-dup & delta**: implement the change-alert of 6.2 as a real script with a state file.
- **Second target side-by-side**: render two companies on one dashboard for comparison.